
Rapport d'activité N°2

Semaine 47 à semaine 49

Rédigé par

CLOÉE HAREAU
STEVEN TRINH
ADRIEN BASCHUNG
WENJIA TANG

Table des matières

1	Travail réalisé	3
1.1	Les noeuds	3
1.2	AstCreator	3
1.3	GraphViz	3
2	Difficultés rencontrées	4
2.1	Débogage	4
2.1.1	Confusion des noms des visiteurs	4
2.1.2	Solution	4
3	Annexes	5

1 Travail réalisé

Après avoir écrit notre grammaire du langage Tiger, nous avons ensuite travaillé sur la construction d'arbres abstraits.

Pour cela, nous avons suivi la trame vue en TP avec la labellisation de règles, l'écriture des classes des nœuds, et les 2 visiteurs AstCreator et GraphVizVisitor.

1.1 Les noeuds

Pour construire l'arbre abstrait, on représente chaque nœud par une classe à laquelle on associe des attributs représentant les fils du nœud.

1.2 AstCreator

L'AstCreator est le visiteur de l'arbre syntaxique qui nous permet de contruire l'arbre abstrait. En effet, la création de l'arbre abstrait s'effectue en visitant chaque nœud de l'arbre syntaxique pour créer les nœuds de l'arbre abstrait tout en prenant soin de supprimer les nœuds unaires. C'est donc ici que l'on appelle les constructeurs des classes des différents nœuds et que l'on redonne du sens aux nœuds en les renommant.

1.3 GraphViz

GraphViz est le visiteur qui va parcourir l'arbre abstrait pour construire les nœuds graphiquement et nous permettre de visualiser l'AST.

2 Difficultés rencontrées

Pendant la construction d'arbres abstraits, les difficultés rencontrées concernaient principalement le débogage du code.

2.1 Débogage

"AstCreator" est le fichier dans lequel tous les visiteurs de l'arbre syntaxique se situent. Ainsi, ce fichier contient énormément de méthodes, ce qui nous ralentissait dans la recherche d'erreurs.

2.1.1 Confusion des noms des visiteurs

La première et seule erreur que nous avons mise beaucoup de temps à trouver est celle de la confusion des noms des visiteurs. Dans notre grammaire, nous avons un label "SeqExpr" et une règle "seq_expr". La similarité de nom est la raison pour laquelle nous nous sommes trompés quand nous implémentions la méthode "visitSeq_Expr" (nous avons implémenté la règle "seq_expr" dans la méthode liée au "SeqExpr").

2.1.2 Solution

Nous avons détecté cette erreur en testant l'exemple du manuel. En réduisant la taille de cet exemple, nous enlevions au fur et à mesure les autres sources d'erreur possibles. Finalement, nous avons réussi à trouver le problème dans le "AstCreator" : la méthode "VisitSeqExpr". Après la comparaison avec notre grammaire, nous nous sommes rendus compte que cette méthode a bien été confondue avec la méthode "visitSeq_expr". En corrigeant la méthode, nous avons ainsi résolu le problème.

3 Annexes

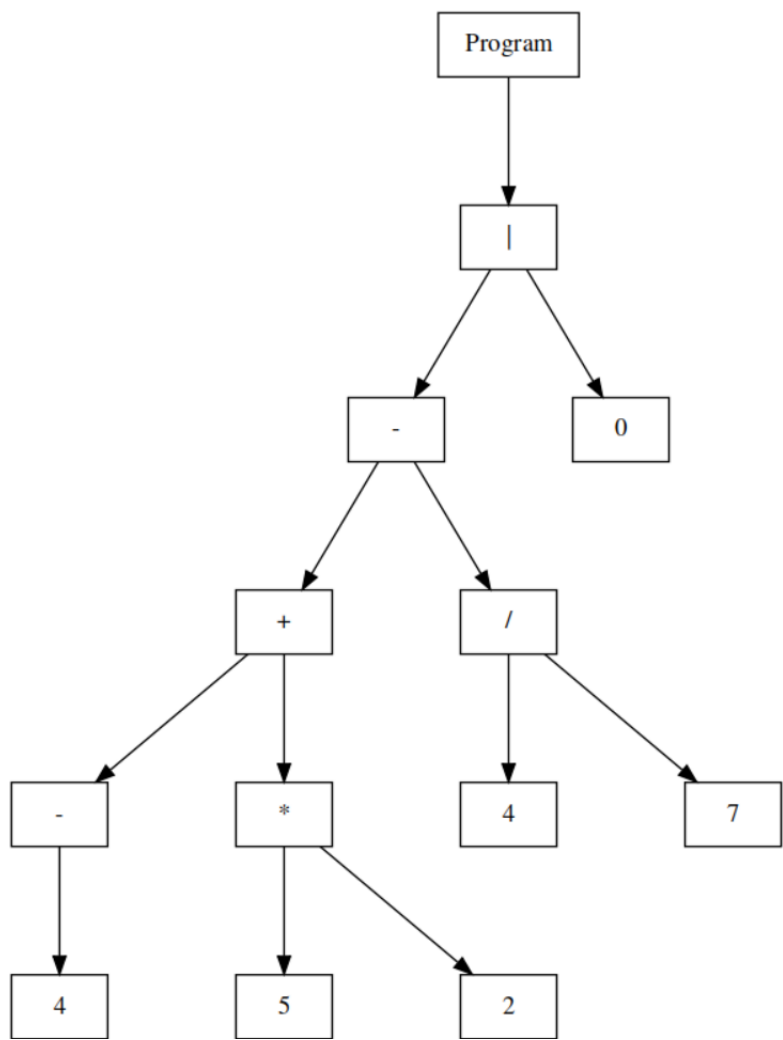


FIGURE 3.1 – ASL de "operations.exp"

1 - 4 + 5 * 2 - 4 / 7 | 0

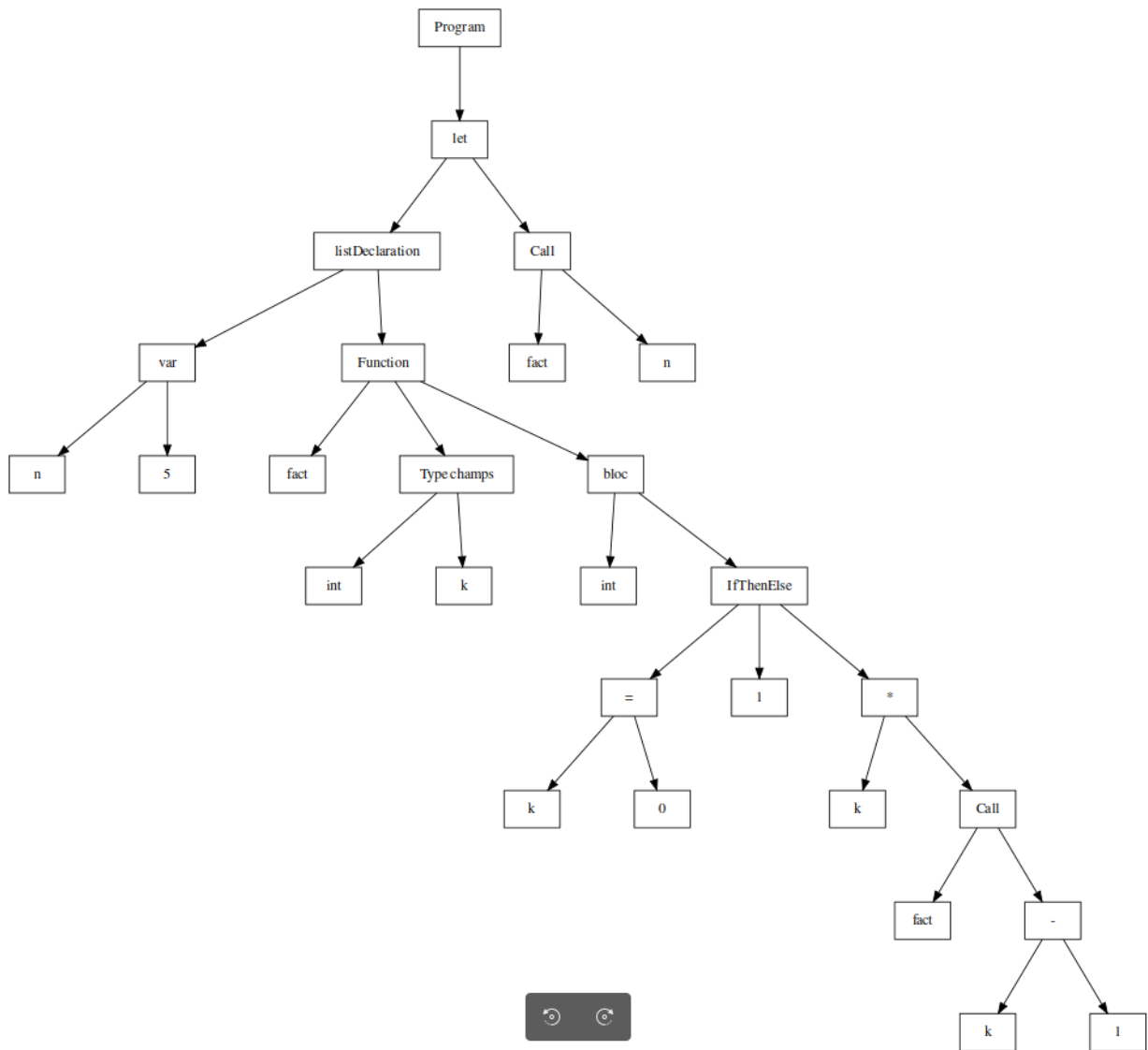


FIGURE 3.2 – ASL de "factoriel_rec.exp"

```

1 let
2   var n := 5
3   function fact(k : int) : int =
4     if k = 0 then (1)
5     else (k * fact(k-1))
6 in
7   fact(n)
8 end

```

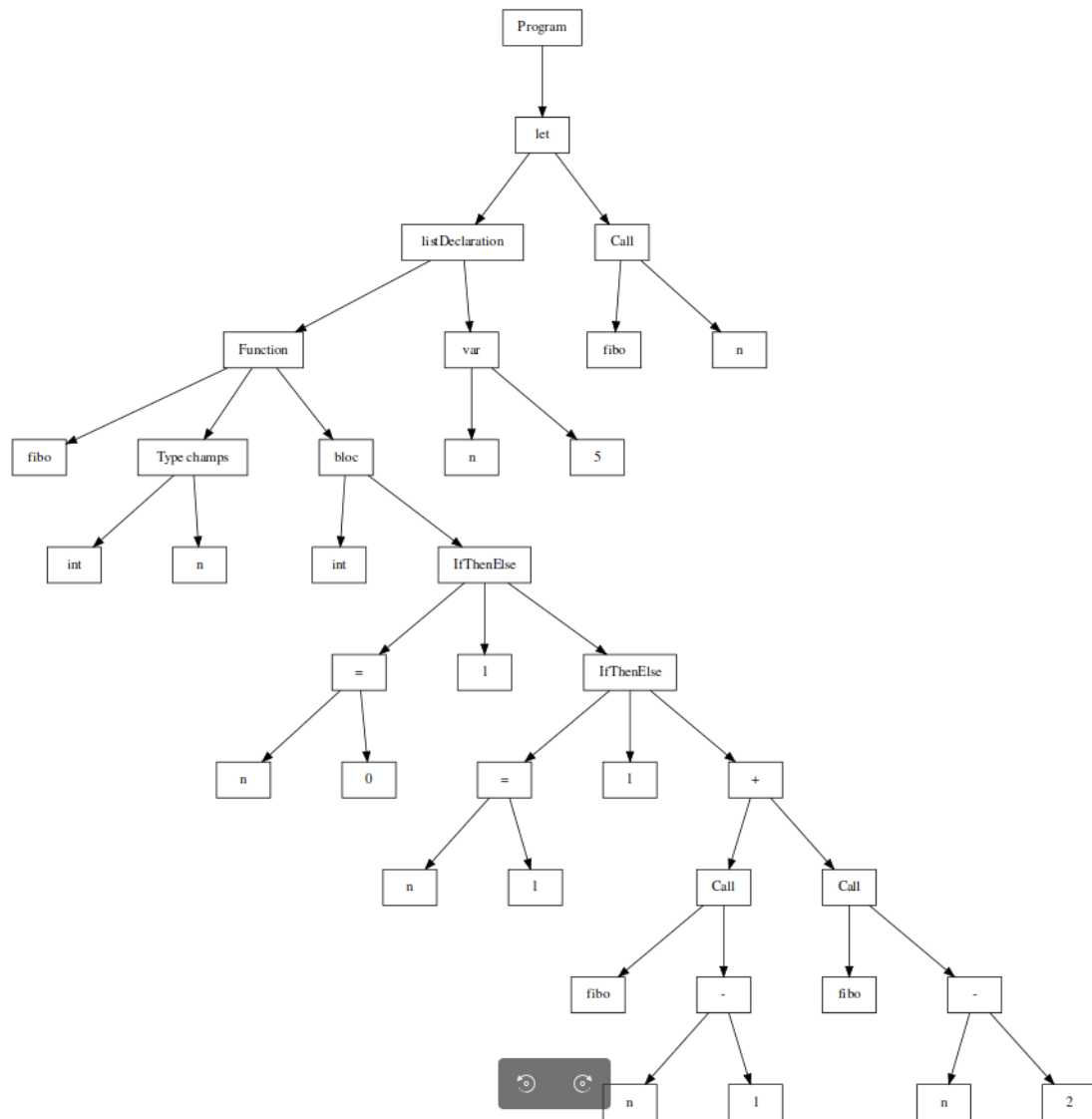


FIGURE 3.3 – ASL de "fibonacci"

```

1 let
2   function fibo(n : int) : int =
3     if n = 0 then (1)
4     else(
5       if n = 1 then(1)
6       else(
7         fibo(n-1) + fibo(n-2)
8       )
9     )
10   var n := 5
11 in
12   fibo(n)
13 end

```